



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт	ИВТИ
Кафедра	МКМ

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ
РАБОТА БАКАЛАВРА**

Направление	01.03.02 Прикладная математика и информатика
	(код и наименование)

Образовательная программа	Математическое моделирование
------------------------------	------------------------------

Форма обучения	очная
	(очная/очно-заочная/заочная)

Тема:	Проектирование и реализация системы централизованного логирования распределённых приложений
-------	--

Студент	A-16-22	Жунусов Е.Е.
	группа	подпись фамилия и инициалы

Руководитель ВКР	старший преподаватель	Шевченко И.В.
	уч. степень должность	подпись фамилия и инициалы

Консультант	уч. степень	должность	подпись	фамилия и инициалы
-------------	-------------	-----------	---------	--------------------

Внешний консультант	уч. степень	должность	подпись	фамилия и инициалы
------------------------	-------------	-----------	---------	--------------------

	организация
--	-------------

«Работа допущена к защите»

Заведующий кафедрой	к.ф.-м.н.	доцент	Зубков П.В.
	уч. степень	звание	подпись фамилия и инициалы

Дата

Москва, 2026



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт	ИВТИ
Кафедра	МКМ

**ЗАДАНИЕ НА ВЫПУСКНУЮ
КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА**

Направление	01.03.02 Прикладная математика и информатика (код и наименование)
--------------------	--

Образовательная программа	Математическое моделирование
----------------------------------	------------------------------

Форма обучения	очная (очная/очно-заочная/заочная)
-----------------------	---------------------------------------

Тема:	Проектирование и реализация системы централизованного логирования распределённых приложений
--------------	--

Студент	A-16-22 группа	Жунусов Е.Е. подпись фамилия и инициалы
----------------	-------------------	---

Руководитель ВКР	старший преподаватель уч. степень должность	Шевченко И.В. подпись фамилия и инициалы
-------------------------	--	--

Консультант	уч. степень должность	подпись фамилия и инициалы
--------------------	-----------------------	-------------------------------

Внешний консультант	уч. степень должность	подпись фамилия и инициалы
----------------------------	-----------------------	-------------------------------

Заведующий кафедрой	к.ф.-м.н. уч. степень	доцент звание	Зубков П.В. подпись фамилия и инициалы
		организация	

Место выполнения работы	ФГБОУ ВО «НИУ «МЭИ»
--------------------------------	---------------------

1. Содержание разделов задания и исходные данные

2. Объём графического материала

Количество листов _____

Количество слайдов в презентации _____

3. Рекомендуемая литература

4. План выполнения выпускной квалификационной работы

№ п\п	Содержание разделов	Срок выполнения	Трудоёмкость, %
I.			
II.			
III.			
IV.	Оформление ВКР		

5. Консультации по ВКР (консультант)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дата

6. Консультации по ВКР (консультант 2)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дата

7. Консультации по ВКР (внешний консультант)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дата

Дата выдачи задания: 09.02.2026
дата

Студент:

<u>Жунусов Евгений Ерланович</u>	_____	_____
фамилия, имя, отчество	подпись	дата

Руководитель ВКР:

<u>Шевченко Иван Владимирович</u>	_____	_____
фамилия, имя, отчество	подпись	дата

Примечания:

1. Задание брошюруется вместе с выпускной работой после титульного листа (страницы задания имеют номера 2, 3, 4, 5).
2. Отзыв руководителя, рецензия(и), отчет о проверке на объем заимствований и согласие студента на размещение работы в открытом доступе вкладываются в конверт (файловую папку) под обложкой работы.

АННОТАЦИЯ

В работе разработан сервер централизованного логирования тренажёрных сессий. Сервер принимает сигналы и события через WebSocket, записывает их пакетами в ClickHouse, возвращает по REST API и архивирует в JSON Lines с gzip-сжатием. Практическая часть реализована на Go и запускается через Docker Compose. После устранения таймаута WebSocket проведён длительный профиль продолжительностью один час: отправлено и полностью сохранено 17 995 850 сигналов и 719 665 событий, средняя скорость составила 4999 сигналов в секунду, клиентских ошибок не зафиксировано. Для сессии объёмом около 1,97 млн записей формирование двух архивов заняло 19,94–29,12 секунды.

Annotation

The thesis presents a centralized logging server for training sessions. The server receives signals and events through WebSocket, writes them to ClickHouse in batches, exposes REST API access, and archives completed sessions as gzip-compressed JSON Lines files. After correcting the WebSocket timeout, a one-hour load profile sent and fully persisted 17,995,850 signals and 719,665 events at an average rate of 4,999 signals per second with no client errors. Creating two archive files for a session containing approximately 1.97 million records took 19.94–29.12 seconds.

Ключевые слова: централизованное логирование, WebSocket, REST API, ClickHouse, Go, буферизация, архивирование, нагрузочное тестирование.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	9
1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ	12
1.1. Источник данных	12
1.2. Типы хранимых данных	12
1.3. Функциональные требования	13
1.4. Нефункциональные требования	14
1.5. Критерии проверки	14
1.6. Выводы по первой главе	15
2. ВЫБОР ТЕХНИЧЕСКОГО РЕШЕНИЯ	16
2.1. Готовая платформа или отдельный сервис	16
2.2. Выбор транспорта	16
2.3. Выбор хранилища	17
2.4. Выбор формата архива	18
2.5. Выбор языка и библиотек	18
2.6. Выводы по второй главе	19
3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ	20
3.1. Архитектура	20
3.2. Поток обработки сообщения	21
3.3. WebSocket и REST API	21
3.4. Модель данных	22
3.5. Буферизация	23
3.6. Архивирование	23
3.7. Удаление	24
3.8. Конфигурация и запуск	24
3.9. Безопасность	25
3.10. Выводы по третьей главе	25
4. РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ	26
4.1. Структура проекта	26
4.2. Запуск сервера	26

4.3. Клиент ClickHouse	27
4.4. Обработка WebSocket	28
4.5. Реализация буферов	29
4.6. Пакетная запись	30
4.7. Чтение данных	31
4.8. Создание и чтение архива	31
4.9. Удаление сессии	33
4.10. Генератор нагрузки	33
4.11. Выводы по четвёртой главе	34
5. ТЕСТИРОВАНИЕ СИСТЕМЫ	35
5.1. Тестовая среда	35
5.2. Функциональная проверка	35
5.3. Методика нагрузочного теста	36
5.4. Результаты нагрузки	37
5.5. Причина потери данных и проверка исправления	39
5.6. Архивирование и сжатие	40
5.7. Обнаруженные ограничения	42
5.8. Предлагаемые изменения	43
5.9. Выводы по пятой главе	44
ПРИЛОЖЕНИЕ 1. ПРИМЕРЫ WEBSOCKET-СООБЩЕНИЙ	46
ПРИЛОЖЕНИЕ 2. КОМАНДЫ ФУНКЦИОНАЛЬНОЙ ПРОВЕРКИ	47
ПРИЛОЖЕНИЕ 3. ФРАГМЕНТЫ СХЕМЫ CLICKHOUSE	48
ПРИЛОЖЕНИЕ 4. ЖУРНАЛ ЧАСОВОГО НАГРУЗОЧНОГО ТЕСТА	49
ЗАКЛЮЧЕНИЕ	50
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	52

ВВЕДЕНИЕ

Тренажёрная система во время работы создаёт два связанных потока данных. Первый поток содержит значения параметров модели: напряжение, ток, мощность, частоту и другие числовые величины. Второй поток содержит события: действия оператора, уведомления, аварии и системные сообщения. Эти данные нужны не только разработчику для поиска ошибок. По ним инструктор может восстановить ход тренировки и понять, какое действие привело к изменению состояния модели. Такой подход соответствует общей задаче обработки больших потоков данных в информационных системах [1].

Хранить такие данные только на компьютере тренажёра неудобно. Локальный файл может быть удалён, перезаписан или остаться на недоступной машине. Кроме того, при нескольких одновременно работающих тренажёрах приходится вручную собирать файлы и сопоставлять их по времени. Поэтому в работе рассматривается отдельный сервер логирования. Тренажёр передаёт ему сообщения во время сессии, а сервер сохраняет их и предоставляет единый интерфейс доступа. Выделение самостоятельного сервиса также уменьшает связанность компонентов распределённого приложения [2].

Особенность задачи состоит в сочетании высокой частоты сигналов и сравнительно редких событий. В длительном испытании один экземпляр тренажёра передавал до 500 сигналов и 20 событий в секунду. При десяти одновременных сессиях суммарная скорость составила 4999 сигналов в секунду и сохранялась в течение одного часа. Записывать каждое сообщение отдельным SQL-запросом нецелесообразно, поэтому сервер накапливает записи в памяти и отправляет их в ClickHouse пакетами.

Ещё одна часть задачи связана с жизненным циклом данных. Сразу после тренировки логи должны быстро читаться из активной базы. Позже их можно перенести в сжатые файлы, освободив место в ClickHouse. После окончания срока

хранения сессия должна удаляться полностью: из таблиц, архива и внутренних буферов сервера.

Цель работы — разработать сервер централизованного логирования, который принимает потоковые данные тренажёрной системы, сохраняет их по сессиям, предоставляет REST API для чтения, архивирует завершённые сессии и удаляет их по запросу.

Для достижения цели решены следующие задачи:

1. Определены типы данных и сценарии их использования.
2. Выбраны транспортный протокол, база данных и формат архива.
3. Спроектированы WebSocket API, REST API и схема таблиц.
4. Реализован сервер на языке Go.
5. Реализована пакетная запись сигналов и событий в ClickHouse.
6. Реализованы архивирование в JSON Lines с gzip-сжатием и чтение из архива.
7. Подготовлено контейнерное окружение Docker Compose.
8. Разработан генератор нагрузки для нескольких одновременных сессий.
9. Проведены функциональные проверки и четыре нагрузочных профиля, включая часовое испытание с высокой частотой сообщений.

Объект исследования — процесс сбора и хранения журналов распределённого приложения. Предмет исследования — программные решения для потокового приёма, пакетной записи, чтения, архивирования и удаления логов тренажёрных сессий.

В работе применены анализ требований, сравнение технических вариантов, проектирование API и схемы данных, реализация программного прототипа и экспериментальная проверка. Результаты тестов сравнивались по числу отправленных и сохранённых записей, фактической скорости передачи, потреблению памяти, размеру данных в ClickHouse, времени архивации и размеру gzip-архивов.

Практический результат работы — проект `logflow`, состоящий из Go-сервера, миграции ClickHouse, Docker-конфигурации и утилиты нагрузочного тестирования. Код проекта содержит около 1,7 тыс. строк с учётом служебных файлов и тестового генератора.

1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ

1.1. Источник данных

Источником логов является тренажёрное приложение. В рамках одной тренировки пользователь может несколько раз перезапустить модель, поэтому одной временной отметки недостаточно. Каждая запись содержит ``session_id``, который связывает её с тренировкой, и ``restart_id``, который отделяет один запуск модели от другого.

Для сигналов используется модельное время ``model_time``. Оно не совпадает с системным временем компьютера: модель можно остановить, ускорить или перезапустить. При анализе тренировки записи сортируются сначала по ``restart_id``, затем по ``model_time``.

Событийные данные поступают реже сигналов, но содержат контекст действий оператора. В поле ``data`` хранится JSON-строка с дополнительными атрибутами. Например, событие может содержать область тренажёра, тег элемента, текст сообщения и текущее значение параметра.

Объём одной сессии зависит от частоты сигналов и длительности тренировки. Если один тренажёр передаёт 500 сигналов в секунду в течение часа, сервер получит 1,8 млн сигнальных записей. При десяти одновременно работающих тренажёрах это уже 18 млн записей за час. Поэтому схема должна быть рассчитана не на единичные сообщения, а на длительный последовательный поток.

Для анализа важна связь двух типов данных. Событие само по себе показывает действие, но не показывает реакцию модели. Сигналы показывают изменение параметров, но не объясняют причину. Сопоставление по ``session_id``, ``restart_id`` и ``model_time`` позволяет восстановить последовательность: действие оператора, возникшее событие и изменение значений модели.

1.2. Типы хранимых данных

В системе используются три сущности:

10. `Session` — метаданные тренировки: идентификатор, время начала, время окончания и статус.
11. `SignalLog` — числовое значение параметра модели.
12. `EventLog` — действие пользователя или системное событие.

Разделение сигналов и событий сделано намеренно. Сигнал всегда содержит числовое значение и имеет стабильный набор полей. Событие может иметь разный состав дополнительных данных. Если хранить оба типа в одной таблице, большая часть столбцов будет пустовать, а запросы станут менее понятными.

1.3. Функциональные требования

Сервер должен выполнять полный цикл работы с данными: принять сообщение, сохранить его, вернуть по запросу, перенести в архив и удалить. Соответствие требований компонентам проекта показано в таблице 1.1.

Таблица 1.1 - Соответствие требований и компонентов реализации

Требование	Компонент	Результат
Потоковый приём	handler.wsStream	WebSocket-соединение на одну сессию
Разделение данных	SignalLog и EventLog	Отдельные модели, буферы и таблицы
Пакетная запись	service.flushSignals / flushEvents	Порог 500 записей, таймер 1 с
Чтение	GET signal-logs / event-logs	limit, offset, restart_id для активных данных
Архивирование	ArchiveSession	Два файла JSONL.GZ на сессию
Удаление	DeleteSession	Партиции, метаданные, файлы и буферы

Важное правило целостности: идентификатор сессии берётся из URL WebSocket-соединения. Поле `session\_id` внутри сообщения перезаписывается сервером. Благодаря этому клиент не может случайно отправить запись в другую сессию.

1.4. Нефункциональные требования

К системе предъявлены следующие требования:

13. Одновременная работа нескольких WebSocket-соединений.
14. Пакетная запись вместо отдельного SQL-запроса на каждое сообщение.
15. Возможность запуска на другой машине без ручной установки компонентов.
16. Отделение сетевой логики, бизнес-логики и доступа к базе данных.
17. Сохранение доступа к данным после архивирования.
18. Проверка API-ключа для операций чтения и управления.
19. Понятная диагностика ошибок через серверный журнал.

Текущая версия является прототипом. Она не гарантирует доставку при аварийном завершении процесса, поскольку незаписанные буферы находятся в оперативной памяти. Это ограничение учитывается при оценке результатов.

Под производительностью в этой работе понимается не максимальное число сообщений, которое теоретически может принять Go-сервер. Проверяется более практический показатель: сколько сообщений генератор смог отправить за заданное время и сколько из них затем найдено в ClickHouse. Такой критерий одновременно учитывает сеть, обработчик, блокировки, буферизацию и запись в базу.

1.5. Критерии проверки

Работоспособность считается подтверждённой, если:

20. Сервер запускается вместе с ClickHouse через Docker Compose.
21. Маршрут `/health` возвращает код 200.
22. WebSocket-клиент передаёт сигналы и события нескольких сессий.
23. Количество записей в ClickHouse совпадает с количеством успешно отправленных сообщений.

24. REST API возвращает данные с фильтрацией и постраничными параметрами.
25. После архивирования создаются два gzip-файла, а чтение через REST API продолжает работать.
26. После удаления исчезают таблицы данных конкретной сессии, метаданные и архивные файлы.

1.6. Выводы по первой главе

Основной единицей системы является тренировочная сессия. Все операции выполняются по `session\_id`: подключение, запись, чтение, архивирование и удаление. Поток разделён на сигналы и события, а порядок внутри тренировки задают `restart\_id` и `model\_time`. Эти решения напрямую отражены в моделях Go и таблицах ClickHouse.

2. ВЫБОР ТЕХНИЧЕСКОГО РЕШЕНИЯ

2.1. Готовая платформа или отдельный сервис

Elasticsearch, Loki и Graylog решают общую задачу централизованного сбора журналов. Однако проекту требуется не только поиск текста. Необходимо принимать числовые сигналы, группировать их по тренировке, учитывать перезапуски модели и архивировать данные всей сессии одной операцией.

Использование готовой платформы добавило бы несколько компонентов и усложнило демонстрацию. Поэтому выбран отдельный сервис с небольшим API. Он не заменяет промышленную платформу наблюдаемости, но реализует нужный для тренажёра цикл данных без лишних функций.

Отказ от готовой платформы также позволяет показать собственную реализацию ключевых механизмов: разбор сообщений, управление сессиями, пакетную вставку, перенос данных между активным и архивным слоями. Для дипломной работы это важнее, чем настройка большого набора готовых компонентов.

2.2. Выбор транспорта

Рассматривались обычные HTTP-запросы, gRPC и WebSocket. Сравнение приведено в таблице 2.1.

Таблица 2.1 - Сравнение способов передачи логов

Вариант	Преимущества	Недостатки	Решение
HTTP POST	Простая реализация	Запрос на каждую запись или клиентский буфер	Не выбран
gRPC stream	Строгая схема, эффективный поток	Генерация кода и дополнительная настройка	Не выбран
WebSocket	Постоянное соединение, JSON, поддержка ping/pong	Нужно управлять долгим соединением	Выбран

WebSocket выбран потому, что тренировка естественно представляется долгоживущим соединением. Клиент открывает соединение при начале сессии, передаёт сообщения и завершает его командой ``session_end``. Новый HTTP-запрос на каждую запись не создаётся. Формат сообщений остаётся простым JSON, что облегчает интеграцию с тренажёром [3].

REST используется отдельно для операций, которым не требуется постоянное соединение: чтения логов, архивирования и удаления.

2.3. Выбор хранилища

PostgreSQL подходит для метаданных и транзакционных операций, но поток сигналов в этой работе имеет аналитический характер: данные в основном добавляются и читаются большими диапазонами. ClickHouse рассчитан на такие вставки и выборки [4].

В ClickHouse записи одной таблицы физически организованы частями. Частые одиночные вставки создают много небольших частей и увеличивают нагрузку на фоновые слияния. По этой причине выбор ClickHouse связан с обязательной буферизацией: база эффективна только тогда, когда приложение передаёт ей блоки данных, а не каждое значение отдельно.

В проекте полезны следующие возможности ClickHouse:

27. Движок ``MergeTree`` для таблиц логов.
28. Колоночное хранение повторяющихся полей.
29. ``LowCardinality(String)`` для элементов, параметров и типов событий.
30. Партиционирование по ``session_id``.
31. Удаление данных сессии командой ``DROP PARTITION``.
32. HTTP-интерфейс, позволяющий работать без отдельного драйвера.

Последний пункт упростил прототип, но имеет цену: SQL-запросы формируются вручную, а ответы ``SELECT`` разбираются из TSV. В промышленной версии безопаснее использовать официальный драйвер ClickHouse с параметризованными запросами.

Партиционирование по каждой сессии удобно для дипломного сценария, где удаление выполняется целиком. Однако при очень большом числе коротких сессий количество партиций может стать избыточным. Для промышленной установки пришлось бы оценить альтернативу: партиционирование по месяцу или дню с дополнительным условием удаления по ``session_id``. В текущих тестах число сессий невелико, поэтому простота удаления важнее.

2.4. Выбор формата архива

Архив должен читаться последовательно и не требовать загрузки всего набора данных при записи. Поэтому каждая запись сериализуется как отдельный JSON-объект в формате JSON Lines на основе формата JSON [9]. Файл сжимается gzip [10].

Преимущества решения:

- 33. Архив можно создавать потоково.
- 34. Повреждение одной строки не меняет структуру всего файла.
- 35. Формат не зависит от ClickHouse.
- 36. Файл можно распаковать стандартными средствами.
- 37. Повторяющиеся имена полей хорошо сжимаются.

Недостаток — отсутствие индекса. В текущей версии для выдачи данных архив читается полностью.

Другим вариантом мог быть Parquet. Он лучше подходит для выборочного чтения столбцов и обычно эффективнее по размеру, но требует дополнительной библиотеки и более сложного кода. JSON Lines выбран как понятный промежуточный вариант: архив можно проверить вручную командой ``gzip -dc``, а его структура совпадает с моделями API.

2.5. Выбор языка и библиотек

Сервер реализован на Go 1.22 [5]. Стандартная библиотека предоставляет HTTP-сервер, контексты, таймеры, синхронизацию и работу с gzip. Для

маршрутизации используется `chi` [6], для WebSocket — `gorilla/websocket` [7]. Контейнерный запуск основан на Docker [8].

В проекте всего две внешние Go-зависимости. Это упрощает сборку и уменьшает размер контейнера. Многостадийный Dockerfile компилирует статический бинарный файл, после чего копирует его в образ Alpine.

2.6. Выводы по второй главе

Выбранная схема состоит из WebSocket для входящего потока, REST API для управления, ClickHouse для активных данных и gzip-файлов для архива. Решение компактно и соответствует масштабу дипломного проекта. Его ограничения — буферы в памяти, ручное формирование SQL и последовательное чтение архива — рассматриваются в пятой главе.

3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1. Архитектура

Система разделена на четыре слоя:

- 38. `handler` принимает HTTP- и WebSocket-запросы.
- 39. `service` управляет сессиями, буферами, архивированием и удалением.
- 40. `repository` формирует запросы к ClickHouse.
- 41. `db` выполняет HTTP-запросы к ClickHouse и разбирает ответы.

Общая схема показана на рисунке 3.1.

Общая архитектура системы централизованного логирования

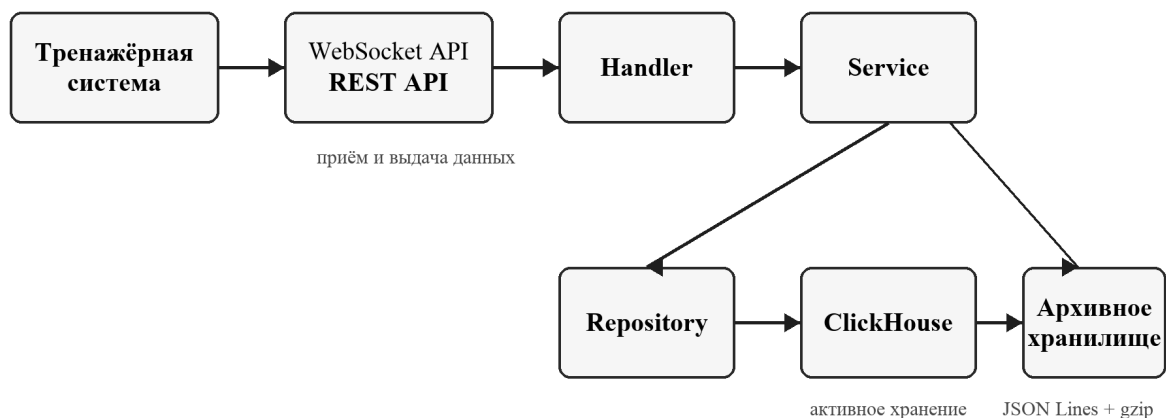


Рисунок 3.1 - Архитектура системы централизованного логирования

Тренажёр не обращается к базе напрямую. Он знает только адрес WebSocket и формат сообщений. Это позволяет изменить способ хранения, не меняя клиентскую часть.

Связь между слоями направлена в одну сторону: `handler` вызывает `service`, `service` — `repository`, `repository` — `db`. Обратных импортов нет. Модели вынесены в отдельный пакет и используются всеми слоями. Такая зависимость упрощает чтение проекта и позволяет в дальнейшем подменить репозиторий в тестах.

3.2. Поток обработки сообщения

Для сигнального сообщения выполняются следующие действия:

42. ``wsStream`` читает текстовый WebSocket-кадр.
43. JSON разбирается в структуру ``WSMessage``.
44. По полю ``type`` выбирается модель ``SignalLog``.
45. ``SessionID`` заменяется значением из URL.
46. ``AddSignalLog`` добавляет запись в буфер нужной сессии.
47. При достижении 500 записей вызывается ``flushSignals``.
48. Репозиторий формирует один ``INSERT`` с несколькими строками.

Для событий путь аналогичен, но используется отдельный буфер и таблица ``event_logs``.

При закрытии соединения возможны два сценария. В штатном сценарии клиент отправляет ``session_end``, сервер сбрасывает оба буфера и возвращается из обработчика. Если соединение оборвано, цикл чтения завершается по ошибке, после чего ``EndSession`` вызывается с ``context.Background()``. Второй вариант защищает от отмены контекста отключившегося клиента, но штатная ветка пока использует ``r.Context()``. Именно это различие проявилось в высоком нагрузочном тесте.

3.3. WebSocket и REST API

WebSocket-маршрут имеет вид:

```
/sessions/{session_id}/stream?api_key=...
```

Поддерживаются сообщения ``signal``, ``event`` и ``session_end``. Сервер отправляет ping каждые 30 секунд и ожидает pong не более 60 секунд.

Маршруты REST API приведены в таблице 3.1.

Таблица 3.1 - Маршруты API

Метод	Маршрут	Назначение
GET	/health	Проверка запуска сервера
GET	/sessions/{id}/stream	WebSocket-приём сообщений
GET	/sessions/{id}/signal-logs	Получение сигналов

GET	/sessions/{id}/event-logs	Получение событий
POST	/sessions/{id}/archive	Создание архива
DELETE	/sessions/{id}	Полное удаление сессии

Для чтения поддерживаются параметры `'limit'`, `'offset'` и `'restart_id'`. Репозиторий ограничивает `'limit'` значением 10 000. Если параметр отсутствует или некорректен, используется 1000.

Ответ для сигналов и событий имеет одинаковую форму: идентификатор сессии, массив `'logs'` и общее число `'total'`. Это удобно для интерфейса с постраничной загрузкой. Для активной базы `'total'` считается отдельным запросом `'count()'`. Для архива поле равно числу объектов, прочитанных из файла.

3.4. Модель данных

Структура таблиц приведена в таблице 3.2.

Таблица 3.2 - Таблицы ClickHouse

Таблица	Основные поля	Движок и ключ
sessions	id, started_at, ended_at, status	ReplacingMergeTree; ORDER BY id
signal_logs	session_id, restart_id, element, parameter, model_time, value	MergeTree; PARTITION BY session_id
event_logs	id, session_id, restart_id, username, event_type, model_time, data	MergeTree; PARTITION BY session_id

Таблицы `'signal_logs'` и `'event_logs'` партиционированы по `'session_id'`. Сортировка `'(session_id, restart_id, model_time)'` совпадает с основным способом чтения данных. Таблица `'sessions'` использует `'ReplacingMergeTree'` и хранит статус сессии.

Поле `'id'` событий генерируется функцией `'generateUUIDv4()'` на стороне ClickHouse. Для сигналов отдельный идентификатор не создаётся, поскольку запись определяется сочетанием сессии, перезапуска, элемента, параметра и

модельного времени. Уникальное ограничение не задано: сервер исходит из того, что источник не отправляет дубликаты.

3.5. Буферизация

В сервисе используются две карты:

```
map[uint64][]model.SignalLog
map[uint64][]model.EventLog
```

Ключом является ``session_id``. Доступ защищён отдельными ``sync.Mutex``, поэтому запись сигналов не блокирует работу с событиями.

Сброс выполняется:

49. При накоплении 500 записей.
50. По таймеру один раз в секунду.
51. При завершении WebSocket-сессии.
52. Перед чтением логов.
53. Перед архивированием.

Перед записью данные копируются во временный срез, а исходный буфер очищается. Блокировка освобождается до обращения к ClickHouse, поэтому сетевой запрос не удерживает mutex.

Одновременно для одной сессии могут сработать пороговый и периодический flush. Mutex не позволяет двум операциям забрать один и тот же срез. Первая операция очищает буфер, вторая видит нулевую длину и завершается. Новые сообщения, поступившие во время SQL-запроса, попадают уже в новый буфер и будут записаны следующей операцией.

3.6. Архивирование

Архивирование запускается запросом ``POST /sessions/{id}/archive``. Сначала сервис сбрасывает оба буфера. Затем данные читаются порциями по 5000 записей и записываются в два файла:

```
signal_logs_{session_id}.jsonl.gz
event_logs_{session_id}.jsonl.gz
```

После создания файлов статус в оперативной памяти меняется на ``archived``. Обновление статуса в ClickHouse и удаление партиций выполняются в фоновой

горутине. Это уменьшает время ответа API, но создаёт промежуток, в котором архив уже доступен, а активные записи ещё удаляются.

Статус дополнительно хранится в карте ``statuses``. Это сделано из-за особенностей мутаций ClickHouse: команда ``ALTER TABLE ... UPDATE`` может применяться не мгновенно. Сразу после архивирования сервис уже должен читать файл, поэтому он не ждёт обновления таблицы. После перезапуска процесса карта пуста, и статус снова определяется запросом ``GetSession``.

3.7. Удаление

Удаление состоит из четырёх шагов:

54. Удаление партиций сигналов и событий.
55. Удаление строки из ``sessions``.
56. Удаление двух архивных файлов.
57. Очистка карт буферов и статусов в памяти.

Единицей удаления остаётся ``session_id``. Данные других тренировок не затрагиваются.

3.8. Конфигурация и запуск

Параметры сервера задаются переменными окружения: ``SERVER_PORT``, ``API_KEY``, ``CH_ADDR``, ``CH_DB``, ``CH_USER``, ``CH_PASSWORD``, ``ARCHIVE_DIR``.

Docker Compose запускает два сервиса. ClickHouse публикует HTTP-порт 8123 и нативный порт 9000. ``logflow`` публикует порт 8080 и подключает отдельный том ``/archives``. При старте сервер делает до десяти попыток подключения к ClickHouse с паузой три секунды, затем применяет миграцию ``001_init.sql``.

Данные ClickHouse и архивы находятся в разных Docker volumes. Пересоздание контейнера не удаляет их, пока не используется ``docker compose down -v``. Миграция содержит ``CREATE TABLE IF NOT EXISTS``, поэтому повторный запуск не создаёт дубликаты таблиц.

3.9. Безопасность

Все маршруты, кроме `/health`, проверяют API-ключ. Для REST он передаётся в `X-API-Key` или `Authorization: Bearer`. Для WebSocket допускается query-параметр.

Это базовая защита. В текущей версии нет TLS, ролей пользователей и аудита административных действий. Кроме того, `CheckOrigin` всегда возвращает `true`. Для закрытой демонстрационной среды это допустимо, но для внешнего развёртывания требует изменения.

3.10. Выводы по третьей главе

Архитектура повторяет фактическую структуру проекта и не содержит лишних промежуточных компонентов. Основные решения — отдельные буферы, пакетный `INSERT`, партиции по сессии и два архивных файла — направлены на конкретные операции дипломной системы.

4. РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ

4.1. Структура проекта

Практическая часть расположена в каталоге `logflow3`. Назначение основных файлов приведено в таблице 4.1.

Таблица 4.1 - Назначение файлов проекта

Файл	Назначение
cmd/server/main.go	Запуск сервера и graceful shutdown
internal/handler/handler.go	Маршруты, API-ключ, WebSocket
internal/service/service.go	Буферы, архивирование, удаление
internal/repository/repository.go	SQL-запросы к ClickHouse
internal/db/db.go	HTTP-клиент ClickHouse и миграции
internal/model/model.go	Структуры сообщений и ответов
cmd/loadtest/main.go	Генератор нагрузки
migrations/001_init.sql	Создание трёх таблиц

Точка входа содержит только сборку зависимостей и запуск сервера. Работа с HTTP не смешивается с SQL, а архивирование не находится в обработчиках маршрутов.

4.2. Запуск сервера

Функция `main` последовательно выполняет:

58. `config.Load()` — чтение переменных окружения.
59. `db.Connect()` — подключение к ClickHouse.
60. `db.RunMigrations()` — создание таблиц.
61. `repository.New()` — создание слоя данных.
62. `service.New()` — запуск сервисного слоя и фонового flush-цикла.
63. `handler.New()` — настройку маршрутов и API-ключа.
64. `ListenAndServe()` — запуск HTTP-сервера.

Для корректного завершения обрабатываются ``SIGINT`` и ``SIGTERM``. Сервер получает 30 секунд на завершение активных запросов.

HTTP-сервер задаёт ``ReadTimeout`` и ``WriteTimeout`` по 30 секунд. Middleware с ограничением 60 секунд применяется к health-check и REST-маршрутам, но исключён из WebSocket-маршрута. Это изменение было внесено после первоначального высокого профиля, в котором общий HTTP timeout отменял контекст соединения на 60-й секунде. Долгоживущее соединение после upgrade управляется библиотекой ``gorilla/websocket`` и механизмом ping/pong.

Обработчик сигнала завершения находится в отдельной горутине. После получения сигнала создаётся контекст с таймаутом 30 секунд и вызывается ``srv.Shutdown``. Новые соединения перестают приниматься, а действующие обработчики получают время на завершение. При этом фоновый ``flushLoop`` не имеет механизма остановки, поскольку сервис не реализует метод ``Close``. Для прототипа процесс завершается вместе с приложением, но в более строгой реализации цикл нужно завершать через канал или контекст.

4.3. Клиент ClickHouse

``CHClient`` использует HTTP-интерфейс ClickHouse. Метод ``do`` создаёт POST-запрос, передаёт имя базы и пользователя в query-параметрах и помещает SQL в тело запроса.

Для команд без результата используется ``Exec``. Для выборки метод ``Query`` добавляет ``FORMAT TSV``, разбивает ответ по строкам и табуляциям. Это решение занимает мало кода, но переносит преобразование типов в репозиторий.

При старте ``Connect`` проверяет ``/ping``. В реальном запуске 9 июня 2026 года первые две попытки завершились отказом в соединении, поскольку контейнер ClickHouse ещё запускался. Третья попытка прошла успешно, после чего миграции были применены автоматически. Этот сценарий подтвердил необходимость повторных попыток.

Миграционный механизм читает файл целиком, пропускает строки-комментарии и считает выражение завершённым, если строка заканчивается

точкой с запятой. Для текущей миграции этого достаточно. Такой разбор не поддерживает сложный SQL с точками с запятой внутри строк или процедур, но в схеме используются только три простых `CREATE TABLE`.

Метод `Ping` выполняет обычный `GET /ping` без передачи имени пользователя и пароля. При конфигурации ClickHouse с обязательной аутентификацией это может отличаться от условий основных запросов. Более точная проверка должна использовать тот же метод `do`, что и рабочие SQL-команды.

4.4. Обработка WebSocket

Маршрут создаётся в `Handler.Router`. После проверки ключа функция `wsStream` извлекает `session_id` через `chi.URLParam` и переводит HTTP-соединение в WebSocket.

После подключения вызывается `StartSession`. Репозиторий вставляет строку в таблицу `sessions` со статусом `active`. Далее запускается `pingLoop`, а основной цикл читает сообщения.

Перед upgrade запрос уже проходит `authMiddleware`. Сначала ключ ищется в `api_key`, затем в заголовке `X-API-Key`, затем в `Authorization: Bearer`. Сравнение выполняется обычной операцией `key != h.apiKey`. Для единственного статического ключа этого достаточно, однако конфигурация по умолчанию `dev-secret-key` не должна использоваться во внешнем окружении.

Разбор выполняется в два этапа. Сначала читается оболочка:

```
type WSMMessage struct {
    Type string      `json:"type"`
    Data  json.RawMessage `json:"data"`
}
```

Затем поле `Data` преобразуется в `SignalLog` или `EventLog`. Такой вариант проще, чем единая структура с большим количеством необязательных полей.

Некорректное сообщение записывается в серверный лог и пропускается. Соединение не закрывается из-за одной ошибочной записи. Неизвестный `type` также не завершает сессию.

Размер входящего сообщения явно не ограничен. `ReadBufferSize` влияет на внутренний буфер библиотеки, но не задаёт максимальный размер кадра. Для защиты от случайно большого JSON следует вызвать `SetReadLimit`. В текущих тестах сообщения имеют размер в пределах нескольких сотен байт.

Ping отправляется из отдельной горутины, а клиентские сообщения в серверном направлении только читаются. Поэтому конкурентной записи обычных и ping-кадров в проекте нет. Если сервер начнёт отправлять подтверждения или события клиенту, доступ к `websocket.Conn` для записи придётся синхронизировать.

4.5. Реализация буферов

Размер буфера задан константой:

```
const (  
    flushSize      = 500  
    flushInterval = time.Second  
)
```

`AddSignalLog` блокирует `signalMu`, добавляет запись и сразу освобождает mutex. Если размер достиг 500, вызывается `flushSignals`.

В `flushSignals` данные сначала копируются:

```
cp := make([]model.SignalLog, len(s.signalBuf[sessionID]))  
copy(cp, s.signalBuf[sessionID])  
s.signalBuf[sessionID] = s.signalBuf[sessionID][:0]
```

При ошибке `InsertSignalLogs` скопированные записи не возвращаются в буфер. Первоначальный высокий профиль показал, что при отмене контекста это приводило к потере последней порции данных. Исключение `WebSocket` из обычного HTTP timeout устранило конкретную причину сбоя, что подтверждено часовым испытанием, однако возврат данных в буфер при иной ошибке репозитория всё ещё требуется.

Контекст передаётся от `WebSocket`-обработчика в сервис без изменения. Пока соединение активно, это позволяет отменить зависший SQL-запрос вместе с запросом клиента. При завершении сессии такое поведение спорно: финальная запись важнее состояния соединения. Для неё лучше создавать отдельный контекст с небольшим таймаутом, например 10 секунд.

Фоновый `flushLoop` раз в секунду получает список идентификаторов из `signalBuf` и вызывает сброс обоих типов. Здесь есть ещё одно ограничение: сессия, содержащая только события и ни одного сигнала, не попадёт в список. Её события будут сброшены по размеру, при чтении или завершении, но не по периодическому таймеру.

После успешного `flush` в карте остаётся пустой срез. Идентификатор не удаляется, поэтому список известных сессий со временем растёт. При небольшом количестве тренировок это незаметно, но длительно работающий сервер должен удалять пустые элементы после завершения или удаления сессии.

4.6. Пакетная запись

`InsertSignalLogs` формирует один SQL-запрос:

```
INSERT INTO signal_logs
    (session_id, restart_id, element, parameter, model_time, value)
VALUES (...), (...), ...
```

При размере буфера 500 один HTTP-запрос заменяет 500 отдельных вставок. Для профиля с 297 750 сигналами это уменьшает порядок числа запросов с сотен тысяч до сотен.

Числовые значения форматируются через `%f`, то есть с шестью знаками после запятой. Для текущей генерации этого достаточно, но часть исходной точности `float64` может быть потеряна ещё до передачи SQL. При необходимости полного сохранения следует использовать форматирование с подходящим числом значащих цифр или нативный драйвер.

Строки экранируются функцией `chStr`, которая заменяет одинарную кавычку. Это достаточно для текущих тестовых данных, но не является полной заменой параметризованных запросов.

В событиях поле `data` хранится как строка. Репозиторий не проверяет, содержит ли она корректный JSON. Это решение допускает разные структуры событий, но перекладывает проверку на источник. Альтернативой является тип `JSON` или проверка `json.Valid` в обработчике.

4.7. Чтение данных

Получение сигналов выполняется двумя запросами. Первый вычисляет ``count()``, второй возвращает страницу:

```
SELECT session_id, restart_id, element, parameter, model_time, value
FROM signal_logs
WHERE session_id = ...
ORDER BY restart_id, model_time
LIMIT ... OFFSET ...
```

Если передан ``restart_id``, он добавляется в ``WHERE``. Такой же подход используется для событий.

Перед чтением сервис сбрасывает соответствующий буфер. Поэтому REST-клиент видит сообщения, которые были приняты недавно и ещё не попали в периодический flush.

Параметр ``offset`` не имеет верхнего ограничения. Большое смещение заставляет ClickHouse пройти предыдущие строки, поэтому для длинных сессий лучше использовать курсор по ``(restart_id, model_time)`` вместо классической пагинации. Для дипломного API ``limit/offset`` проще и понятнее, но это компромисс.

Если сессия не найдена, репозиторий возвращает пустой массив и ``total=0``, а HTTP-код остаётся 200. Архивный путь, напротив, возвращает внутреннюю ошибку и код 500, если файл отсутствует. Единая политика 404 для неизвестной сессии сделала бы API последовательнее.

4.8. Создание и чтение архива

Функции ``archiveSignals`` и ``archiveEvents`` читают ClickHouse порциями по 5000 строк. Для каждой записи ``json.Encoder`` создаёт отдельную строку, а ``gzip.Writer`` сразу сжимает поток.

При чтении используется ``gzip.NewReader`` и ``json.Decoder``. Внешний формат ответа не меняется: клиент получает ``session_id``, ``logs`` и ``total``.

Текущая реализация не применяет ``limit``, ``offset`` и ``restart_id`` к архиву. Экспериментальный запрос ``limit=10`` для архивной сессии с 29 950 сигналами вернул все 29 950 записей, 3,32 МБ данных. Время ответа составило 0,124

секунды на тестовой машине. Для более крупного архива объём ответа и расход памяти будут расти линейно.

Функции архивации используют ``defer`` для закрытия файла и `gzip-writer`. Если ошибка возникнет во время чтения ClickHouse, частично записанный файл останется с окончательным именем. Надёжнее создавать временный файл с расширением ``.tmp``, закрывать его, а затем атомарно переименовывать. Тогда наличие файла будет означать завершённый архив.

Архивирование сигналов и событий выполняется последовательно. Если первый файл создан, а второй завершился ошибкой, статус не меняется, но на диске остаётся неполный набор. Для восстановления запрос можно повторить: ``os.Create`` перезапишет файлы. Более надёжный вариант — создавать временные файлы и атомарно переименовывать их после успешного завершения обоих этапов.

Операция имеет смешанную синхронно-асинхронную модель. HTTP-клиент синхронно ожидает сброса буферов, чтения всех записей из ClickHouse и полного формирования двух `gzip`-файлов. Только после этого обработчик возвращает ``200 OK`` со статусом ``.archived``.

После формирования файлов сервис запускает фоновую `goroutine`, которая обновляет статус сессии в ClickHouse и удаляет активные партиции. Клиент не ждёт эту часть и не получает отдельного уведомления о её завершении. Ошибки фоновых операций записываются только в серверный журнал.

REST-маршрут архивирования ограничен 60 секундами. Измеренные большие сессии архивировались быстрее этого предела, однако при дальнейшем росте объёма запрос может быть отменён, а на диске может остаться частичный файл. Для промышленного варианта предпочтительна очередь заданий с идентификатором операции и отдельным маршрутом проверки статуса.

4.9. Удаление сессии

`DeleteSessionLogs` выполняет `DROP PARTITION` отдельно для сигналов и событий. Затем `DeleteSession` удаляет метаданные. Сервис удаляет файлы через `os.Remove` и очищает три карты в памяти.

В функциональном тесте удаление архивной сессии заняло 0,015 секунды. После запроса оба файла отсутствовали, а `SELECT count() FROM sessions FINAL WHERE id=1009` вернул 0.

Ошибки `os.Remove` игнорируются. Это делает удаление идемпотентным при отсутствии файла, но одновременно скрывает проблему прав доступа. Корректнее отдельно игнорировать только `os.IsNotExist`, а остальные ошибки возвращать клиенту.

4.10. Генератор нагрузки

Утилита `cmd/loadtest` создаёт отдельную горутину и WebSocket-соединение для каждой сессии. Сигналы отправляются пакетами по 50 сообщений. Частота задаётся флагами `signals-per-sec`, `events-per-sec`, `sessions` и `duration`. В `Makefile` добавлен профиль `loadtest-hour`, который запускает 10 сессий, по 500 сигналов и 20 событий в секунду на сессию в течение одного часа.

Счётчики реализованы через `atomic.Int64`. Каждые пять секунд программа выводит число отправленных сообщений и среднюю скорость. После подключения запускается фоновое чтение WebSocket-кадров: библиотека обрабатывает `ping` сервера и автоматически отправляет `pong`. Без этого долгий тест разрывался по серверному `read deadline` примерно через 60 секунд. После завершения каждой сессии отправляется `session_end`.

Генератор считает ошибкой неудачную отправку по WebSocket. Поэтому клиентские счётчики дополнительно сверяются с ClickHouse. В часовом испытании прирост базы относительно исходных счётчиков точно совпал с 17 995 850 сигналами и 719 665 событиями, отправленными клиентом.

Начало всех сессий немного сдвигается: между запуском горутин есть пауза 100 миллисекунд. Из-за этого последние сессии работают чуть меньше общего значения ``duration``, которое отсчитывается от единого ``startTime``. Поэтому фактическое число сообщений ниже простого произведения частоты, количества сессий и времени. Это не ошибка сервера и учитывается сравнением с фактическими клиентскими счётчиками.

Сигнальный таймер отправляет по 50 сообщений за одно срабатывание. При 500 сигналах в секунду интервал равен 100 миллисекундам. Такой режим создаёт короткие всплески, близкие к поведению клиента, который сам собирает данные небольшими пачками.

4.11. Выводы по четвёртой главе

Реализация соответствует спроектированным слоям и собирается без ошибок командой ``go test ./...``, хотя автоматических тестов пока нет. Долговременная проверка подтвердила корректность новой политики `timeout` и обработки `ping/pong`. Сохраняются ограничения: очистка буфера до подтверждения записи, периодический обход только ``signalBuf``, синхронное формирование архива в HTTP-запросе и отсутствие постраничного чтения архивных данных.

5. ТЕСТИРОВАНИЕ СИСТЕМЫ

5.1. Тестовая среда

Тесты выполнены 9 июня 2026 года на компьютере с macOS 15.7.4 и архитектурой arm64. Docker Desktop предоставлял контейнерам 7,75 ГиБ памяти.

Использовались:

- 65. Go 1.22.
- 66. ClickHouse `24.3-alpine`.
- 67. Образ Alpine 3.19 для сервера.
- 68. Docker Compose из файла проекта.
- 69. API на `localhost:8080`.

Перед каждым нагрузочным профилем таблицы `signal\_logs`, `event\_logs` и `sessions` очищались. Благодаря этому результаты профилей не смешивались.

Контейнеры собирались из текущего состояния каталога `logflow3`. После старта проверялись состояние сервисов и серверный журнал. ClickHouse переходил в состояние `healthy`, а приложение сообщало об успешном применении миграций. Генератор нагрузки запускался с хостовой системы и подключался к опубликованному порту 8080.

Размер данных в ClickHouse определялся по `system.parts`: суммировались `data\_uncompressed\_bytes` активных частей таблиц сигналов и событий. Размер архивов измерялся внутри контейнера `logflow` командой `wc -c /archives/\*.gz`. Такой способ позволяет повторить измерения без дополнительного программного обеспечения.

5.2. Функциональная проверка

Результаты функциональных запросов приведены в таблице 5.1.

Таблица 5.1 - Результаты функциональной проверки

Проверка	Результат	Время / объём
GET /health	HTTP 200	0,0037 с; 16 байт
Запрос без API-ключа	HTTP 401	0,0029 с; 39 байт

Чтение архивной сессии с limit=10	HTTP 200, но возвращён весь архив	0,124 с; 3,32 МБ
DELETE архивной сессии	HTTP 200; файлы и метаданные удалены	0,015 с
go test ./...	Сборка всех пакетов успешна	Автоматических тестов нет
Архивирование большой сессии	HTTP 200; архивы содержат все 1 973 121 записи	19,94–29,12 с; 25,25 МБ

Проверка показала, что API-ключ обрабатывается корректно: запрос без ключа получил код 401 за 0,0029 секунды. Удаление очистило метаданные и архивы. Основное несоответствие обнаружено при чтении архивной сессии: параметры постраничной выдачи игнорируются.

Кроме HTTP-ответа проверялось состояние хранилища. После удаления сессии 1009 оба gzip-файла отсутствовали, а запрос к `sessions FINAL` вернул ноль строк. Это подтверждает, что операция затрагивает не только внешний API, но и все места хранения.

Команда `go test ./...` завершилась успешно для восьми пакетов проекта. Результат подтверждает сборку, но не проверяет поведение: каждый пакет вывел `[no test files]`. Поэтому основная проверка выполнялась интеграционно через запущенные контейнеры.

5.3. Методика нагрузочного теста

Использовались четыре профиля из `Makefile`. Лёгкий профиль проверяет базовый сценарий. Средний увеличивает суммарный поток до 1500 сигналов в секунду. Высокий запускает 10 соединений и целевой поток 5000 сигналов в секунду в течение 60 секунд. Длительный профиль повторяет параметры высокого, но работает один час и проверяет стабильность соединений, памяти и хранилища на большом объёме.

Для каждого профиля фиксировались:

70. Фактическое время работы.
71. Число отправленных сигналов и событий.
72. Средняя скорость передачи.
73. Клиентские ошибки.
74. Число строк в ClickHouse.
75. Размер данных в активной базе.
76. Размер gzip-архивов.

Количество сохранённых строк проверялось отдельными SQL-запросами. Это важно, поскольку отсутствие сетевой ошибки у клиента ещё не доказывает успешную запись.

После каждого теста выполнялся запрос количества строк по каждой сессии. Первоначальный высокий профиль локализовал потерю последней порции одной сессии из-за общего HTTP timeout. После изменения маршрутизации и добавления чтения служебных WebSocket-кадров длительный профиль использовался как регрессионная проверка исправления.

5.4. Результаты нагрузки

Результаты четырёх профилей приведены в таблице 5.2.

Таблица 5.2 - Результаты нагрузочных тестов

Профиль	Сессии	Отправлено сигналов	Скорость, сигн./с	Сохранено	Потеря
Лёгкий	3	5 950	296	5 950	0
Средний	5	44 600	1 486	44 600	0
Высокий	10	297 750	4 961	297 450	300 (0,101%)
1 час	10	≈18 млн	4 999	≈18 млн	0

Фактическая скорость показана на рисунке 5.1.

Фактическая скорость передачи сообщений

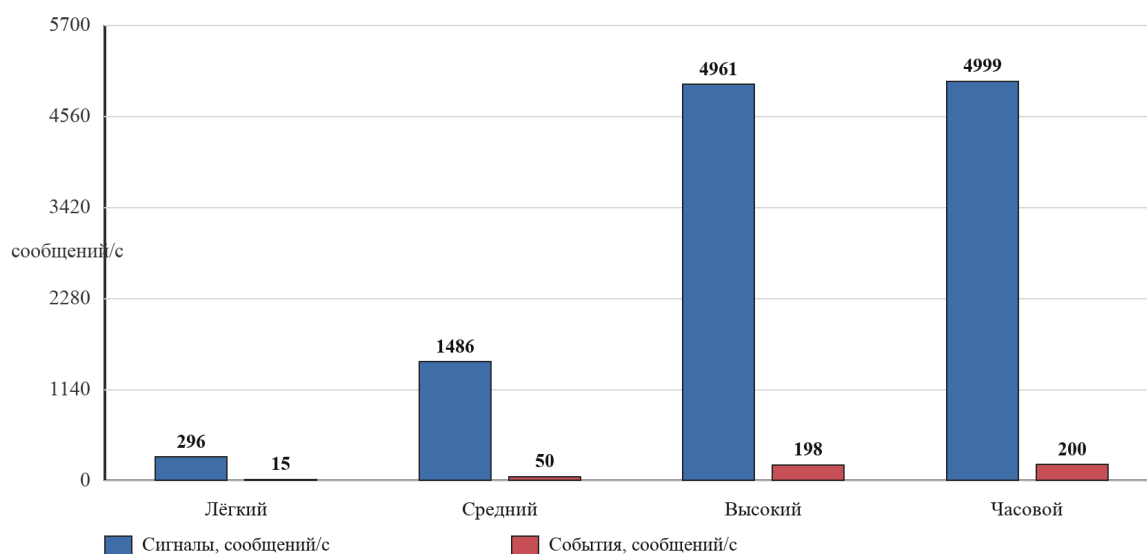


Рисунок 5.1 - Фактическая скорость передачи сообщений

Лёгкий профиль отправил 5950 сигналов и 297 событий. Все записи были найдены в ClickHouse. Средний профиль также завершился без расхождений: 44 600 сигналов и 1489 событий.

В лёгком профиле средняя скорость составила 296 сигналов и 15 событий в секунду. Она близка к заданным 300 и 15 сообщениям в секунду. Небольшое отличие связано со сдвигом запуска горутин и завершением по общему времени.

Средний профиль дал 1486 сигналов и 50 событий в секунду при целевых значениях 1500 и 50. Расхождение по сигналам составило менее одного процента. Серверный журнал не содержал ошибок flush, а число строк полностью совпало с клиентскими счётчиками.

Первоначальный высокий профиль передал 297 750 сигналов со средней скоростью 4961 сигнал в секунду. Клиент не зафиксировал ошибок, но в ClickHouse оказалось 297 450 сигналов и 11 890 событий. Не сохранились 300 сигналов и 10 событий из-за отмены контекста на границе 60 секунд.

После точечного исправления выполнен длительный профиль. За 3600 секунд отправлено 17 995 850 сигналов и 719 665 событий. Средняя скорость

составила 4999 сигналов и около 200 событий в секунду. Все десять WebSocket-соединений завершились штатно, клиентских ошибок не было.

Полнота сохранения показана на рисунке 5.2.

Доля записей, сохранённых в ClickHouse

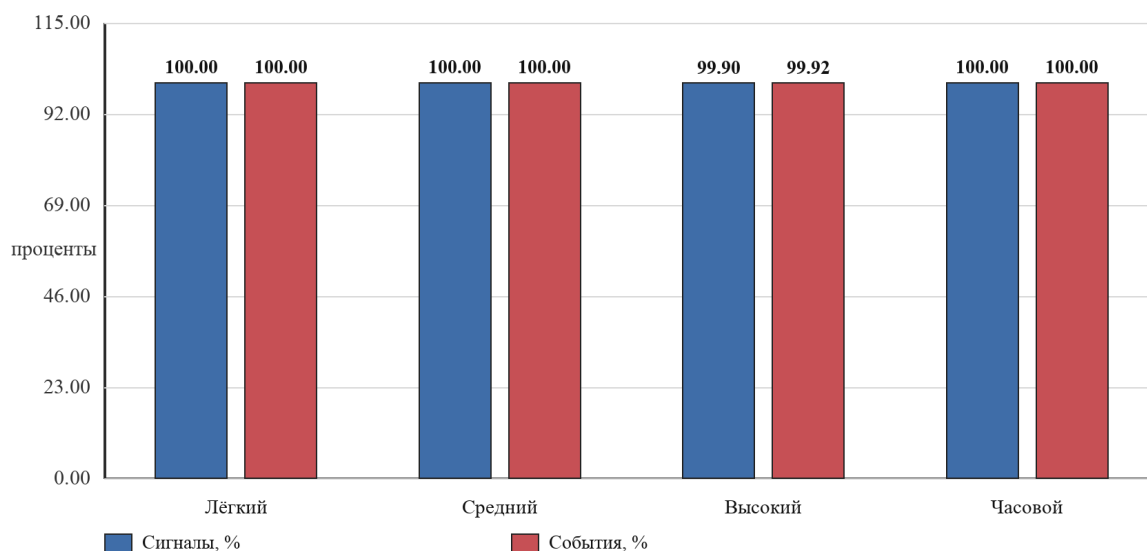


Рисунок 5.2 - Полнота сохранения сообщений

Сверка выполнялась по приросту таблиц относительно исходного состояния. ClickHouse сохранил ровно 17 995 850 новых сигналов и 719 665 новых событий, поэтому расхождение длительного профиля равно нулю. Максимальное наблюдаемое потребление памяти ClickHouse составило 2,67 ГиБ из доступных контейнерам 7,75 ГиБ; сервер Go использовал около 15–20 МиБ.

Длительный профиль подтвердил, что после исправления сервер способен поддерживать целевой поток около 5 тыс. сигналов в секунду в течение часа без накопления ошибок и без роста памяти до критических значений.

5.5. Причина потери данных и проверка исправления

В первоначальном 60-секундном испытании серверный журнал содержал следующие сообщения:

```
ERROR flush signal_logs session=1000: context deadline exceeded
ERROR flush event_logs session=1000: context deadline exceeded
ws: end session=1000 error: context deadline exceeded
```

Причиной был общий ``middleware.Timeout(60 * time.Second)``, который охватывал WebSocket-маршрут. При обработке ``session_end`` контекст одной сессии уже был отменён, а ``EndSession`` передал его в ``flushSignals`` и ``flushEvents``.

Буфер очищался до вызова репозитория. После ошибки записи данные не возвращались, поэтому одна сессия потеряла последнюю порцию: 300 сигналов и 10 событий.

Размер потери не равен стандартному порогу 500, потому что периодический flush продолжал работать во время теста. К моменту завершения в буфере оставалось только то, что поступило после последнего успешного сброса. Для сессии 1000 это были 300 сигналов и 10 событий.

Строка ``response.WriteHeader on hijacked connection`` дополнительно показала, что обычный HTTP timeout пытался сформировать ответ после перевода соединения в WebSocket.

Для устранения обнаруженной причины внесены два точечных изменения:

77. WebSocket-маршрут исключён из 60-секундного middleware; ограничение сохранено для обычных REST-запросов.
78. Нагрузочный клиент постоянно читает соединение, поэтому корректно обрабатывает ping и отправляет pong в длительных сессиях.

Механизм возврата очищенного буфера при произвольной ошибке репозитория пока не реализован и остаётся отдельным направлением повышения надёжности.

Регрессионная проверка продолжалась один час, то есть в 60 раз дольше исходного высокого профиля. Критерии исправления выполнены: клиентские счётчики совпали с числом строк, а сообщения ``context deadline exceeded`` в серверном журнале отсутствовали.

5.6. Архивирование и сжатие

Размеры активных данных и архивов приведены в таблице 5.3.

Таблица 5.3 - Результаты архивирования

Профиль	Несжатые данные, байт	Архивы gzip, байт	Уменьшение
Лёгкий	269 671	84 524	68,7%
Средний	1 846 925	586 483	68,2%
Высокий	12 748 100	3 965 401	68,9%
Часовой	81 247 397	25 251 962	68,9%

Сравнение объёма показано на рисунке 5.3.

Объём данных до и после архивирования

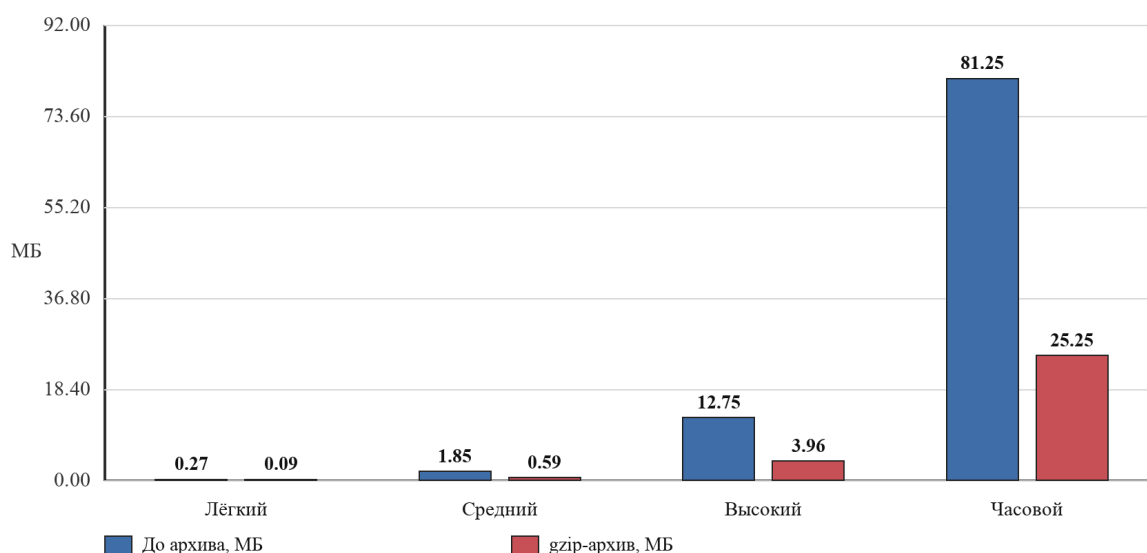


Рисунок 5.3 - Объём данных до и после архивирования

Во всех профилях gzip уменьшил объём примерно на 68–69% относительно логического несжатого размера данных ClickHouse. Для одной сессии длительного профиля 81,25 МБ данных превратились в два архива общим размером 25,25 МБ.

Сравнение не учитывает служебные файлы и метаданные ClickHouse, поэтому его следует рассматривать как оценку данных, а не полный размер каталога базы. Тем не менее результат подтверждает целесообразность архивного слоя.

Коэффициент уменьшения вычислялся по формуле:

$$R = (1 - S_archive / S_source) * 100\%$$

где `S\_source` — суммарный несжатый размер активных частей двух таблиц, а `S\_archive` — размер двух gzip-файлов всех сессий профиля.

Близкий процент во всех четырёх профилях объясняется одинаковой структурой тестовых записей. Имена полей, элементов, параметров и типов событий многократно повторяются, поэтому gzip эффективно использует словарь повторов. Для реальных данных коэффициент может отличаться.

Для оценки ожидания клиента архивировались две часовые сессии. Первая содержала 1 896 850 сигналов и 75 877 событий и была обработана за 19,94 секунды. Вторая содержала 1 897 250 сигналов и 75 871 событие; время ответа составило 29,12 секунды. В обоих случаях HTTP 200 был получен только после появления двух полностью сформированных файлов.

После ответа клиенту обновление статуса и команды `DROP PARTITION` выполнялись асинхронно. По журналу ClickHouse эти три команды заняли 8, 10 и 7 мс соответственно. Отдельного уведомления об окончании фоновой стадии нет; клиент может лишь повторно запросить данные или ориентироваться на ответ `archived`, означающий готовность файлов.

Такая схема уменьшает задержку ответа на время удаления, но не делает само архивирование асинхронным. На больших наборах клиент всё равно ожидает десятки секунд. Если формирование файлов превысит 60 секунд, REST timeout отменит запрос. Поэтому дальнейшая версия должна запускать архивацию как фоновое задание и возвращать идентификатор операции со статусами `queued`, `running`, `completed` и `failed`.

5.7. Обнаруженные ограничения

Итоговый список проблем приведён в таблице 5.4.

Таблица 5.4 - Обнаруженные ограничения и предлагаемые исправления

Проблема	Проявление	Исправление
----------	------------	-------------

Таймаут WebSocket 60 с	В первоначальном тесте отменял финальный flush	Исправлено: timeout применяется только к REST
Буфер очищается до INSERT	Потеря 300 сигналов и 10 событий	Возврат данных в буфер или очистка после успеха
flushLoop читает только signalBuf	События без сигналов не сбрасываются по таймеру	Объединять ID двух карт
Архив читается целиком	limit=10 вернул 29 950 записей	Потоковая пагинация или индекс
Нет автоматических тестов	Регрессии обнаруживаются вручную	Unit- и интеграционные тесты
Синхронное формирование архива	Клиент ожидает 19,94–29,12 с; предел REST — 60 с	Фоновое задание и endpoint состояния

Наиболее важным оставшимся риском является потеря очищенного буфера при ошибке записи. Для эксплуатации также существенна синхронная архивация: она не теряет данные в проведённом опыте, но задержка растёт вместе с размером сессии и ограничена REST timeout.

Таблица разделяет обнаруженные проблемы по месту возникновения. Таймаут относится к HTTP-слою, потеря буфера — к сервисному слою, чтение всего архива — к архивному слою. Это позволяет исправлять их независимо, не переписывая архитектуру проекта целиком.

5.8. Предлагаемые изменения

Для следующей версии проекта целесообразны точечные изменения:

79. Реализовать очередь фоновых заданий архивирования и REST-маршрут получения состояния операции.
80. Изменить `flushSignals` и `flushEvents`: очищать буфер только после успешной вставки либо возвращать данные при ошибке.
81. В `flushLoop` объединять идентификаторы из `signalBuf` и `eventBuf`.

- 82. Обновлять ``ended_at`` при завершении сессии.
- 83. Добавить чтение архива с учётом ``limit``, ``offset`` и ``restart_id``, чтобы большие gzip-файлы не загружались в память целиком.
- 84. Добавить unit-тесты сервисного слоя с подменой репозитория.
- 85. Добавить интеграционный тест, который сравнивает число отправленных и сохранённых записей.
- 86. Перейти на официальный ClickHouse-драйвер для параметризованных вставок.

Добавление брокера сообщений возможно, но для текущего масштаба сначала важнее исправить обработку ошибок и тесты. Это даст больший эффект при меньшем усложнении системы.

Минимальный набор автоматических проверок должен включать:

- 87. Ошибку репозитория во время flush и сохранение записей в буфере.
- 88. Сессию только с событиями и периодический сброс.
- 89. Архив с фильтром ``restart_id``.
- 90. Пагинацию архива с ``limit`` и ``offset``.
- 91. WebSocket-сессию длительностью больше 60 секунд с обязательной сверкой клиентских счётчиков и ClickHouse.
- 92. Повторное архивирование одной сессии.
- 93. Удаление при отсутствии одного из архивных файлов.

Для unit-тестов сервису потребуется интерфейс репозитория вместо конкретного ``*repository.Repository``. Тогда тест сможет подставить реализацию, которая считает вызовы и намеренно возвращает ошибки.

5.9. Выводы по пятой главе

Система успешно обработала лёгкий и средний профили без потерь. Первоначальный высокий профиль обнаружил конфликт WebSocket с HTTP timeout. После точечного исправления часовой профиль сохранил 17 995 850 сигналов и 719 665 событий без расхождений при средней скорости 4999 сигналов в секунду.

Архивирование большой сессии выполняется смешанно: клиент синхронно ожидает формирования двух gzip-файлов 19,94–29,12 секунды, после чего статус и удаление партиций обрабатываются в фоне без отдельного уведомления. Сжатие уменьшило объём на 68,9%. Постраничное чтение и фоновая модель длительных заданий требуют дальнейшей доработки.

ПРИЛОЖЕНИЕ 1. ПРИМЕРЫ WEBSOCKET-СООБЩЕНИЙ

Сигнал:

```
{
  "type": "signal",
  "data": {
    "session_id": 1000,
    "restart_id": 1,
    "element": "H1S3S321",
    "parameter": "U",
    "model_time": 10.2,
    "value": 231.7
  }
}
```

Событие:

```
{
  "type": "event",
  "data": {
    "session_id": 1000,
    "restart_id": 1,
    "username": "ivanovii",
    "event_type": "Alarm",
    "model_time": 11.4,
    "data": "{\"area\":\"Котел\", \"message\":\"Тестовое событие\"}"
  }
}
```

Завершение сессии:

```
{
  "type": "session_end",
  "data": {
    "session_id": 1000
  }
}
```

ПРИЛОЖЕНИЕ 2. КОМАНДЫ ФУНКЦИОНАЛЬНОЙ ПРОВЕРКИ

```
docker compose up -d --build
curl http://localhost:8080/health
make loadtest-light
curl -H "X-API-Key: dev-secret-key" \
    "http://localhost:8080/sessions/1000/signal-logs?limit=10"
curl -X POST -H "X-API-Key: dev-secret-key" \
    "http://localhost:8080/sessions/1000/archive"
curl -X DELETE -H "X-API-Key: dev-secret-key" \
    "http://localhost:8080/sessions/1000"
```

ПРИЛОЖЕНИЕ 3. ФРАГМЕНТЫ СХЕМЫ CLICKHOUSE

```
CREATE TABLE IF NOT EXISTS signal_logs (  
    session_id UInt64,  
    restart_id UInt64,  
    element LowCardinality(String),  
    parameter LowCardinality(String),  
    model_time Float64,  
    value Float64  
)  
ENGINE = MergeTree()  
PARTITION BY session_id  
ORDER BY (session_id, restart_id, model_time);  
CREATE TABLE IF NOT EXISTS event_logs (  
    id UUID DEFAULT generateUUIDv4(),  
    session_id UInt64,  
    restart_id UInt64,  
    username LowCardinality(String),  
    event_type LowCardinality(String),  
    model_time Float64,  
    data String  
)  
ENGINE = MergeTree()  
PARTITION BY session_id  
ORDER BY (session_id, restart_id, model_time);
```


ПРИЛОЖЕНИЕ 4. ЖУРНАЛ ЧАСОВОГО НАГРУЗОЧНОГО ТЕСТА

Сессий: 10

Сигналов/сек на сессию: 500

Событий/сек на сессию: 20

Длительность: 3600 секунд

Сигналов отправлено: 17 995 850

Событий отправлено: 719 665

Клиентских ошибок: 0

Сигналов сохранено: 17 995 850

Событий сохранено: 719 665

Средняя скорость: 4999 сигналов/с и 200 событий/с

Максимальная наблюдаемая память: ClickHouse 2,67 ГиБ; сервер 20 МиБ

Архивация одной сессии: 1 897 250 сигналов + 75 871 событие; 29,12 с; gzip
25 251 962 байта

Результат архивации: HTTP 200; количество строк в двух файлах совпало с
исходными счётчиками

ЗАКЛЮЧЕНИЕ

В работе разработан прототип системы централизованного логирования тренажёрных сессий. Сервер принимает сигналы и события через WebSocket, накапливает их в отдельных буферах и записывает пакетами в ClickHouse. Для чтения, архивирования и удаления реализован REST API.

Практическая часть включает Go-сервис, миграцию базы данных, Dockerfile, Docker Compose и генератор нагрузки. Схема таблиц учитывает идентификатор сессии, перезапуск и модельное время. Партиционирование по ``session_id`` позволяет удалять данные тренировки без построчной обработки.

Функциональные тесты подтвердили запуск окружения, проверку API-ключа, получение данных, архивирование и полное удаление сессии. Первоначальный высокий профиль выявил потерю около 0,1% данных из-за отмены контекста на 60-й секунде. После исключения WebSocket из обычного HTTP timeout и добавления обработки ping/pong проведён часовой профиль: 17 995 850 сигналов и 719 665 событий были сохранены полностью.

Длительный результат подтверждает устойчивую обработку около 5 тыс. сигналов в секунду и показывает, что исправление устранило выявленную причину потери. При этом защита буфера от произвольных ошибок репозитория остаётся необходимой доработкой.

Архивирование в JSON Lines с gzip уменьшило объём большой сессии на 68,9%. Формирование файлов выполняется синхронно относительно REST-клиента и заняло 19,94–29,12 секунды, тогда как обновление статуса и удаление активных партиций выполняются асинхронно после ответа. Отдельного уведомления о завершении фоновой стадии нет.

Таким образом, цель работы достигнута на уровне программного прототипа. Экспериментально проверен полный цикл данных: часовой приём, пакетная запись, чтение, архивирование большого набора и удаление. Четвёртый нагрузочный профиль подтвердил отсутствие потерь и позволил определить

следующий приоритет развития — перевод длительной архивации в модель фоновых заданий с наблюдаемым статусом.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

94. Kleppmann M. Designing Data-Intensive Applications. Sebastopol: O'Reilly Media, 2017. 616 p.
95. Newman S. Building Microservices: Designing Fine-Grained Systems. 2nd ed. Sebastopol: O'Reilly Media, 2021. 616 p.
96. The WebSocket Protocol: RFC 6455 [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обращения: 09.06.2026).
97. ClickHouse Documentation [Электронный ресурс]. URL: <https://clickhouse.com/docs/> (дата обращения: 09.06.2026).
98. The Go Programming Language. Documentation [Электронный ресурс]. URL: <https://go.dev/doc/> (дата обращения: 09.06.2026).
99. Chi router documentation [Электронный ресурс]. URL: <https://github.com/go-chi/chi> (дата обращения: 09.06.2026).
100. Gorilla WebSocket package documentation [Электронный ресурс]. URL: <https://github.com/gorilla/websocket> (дата обращения: 09.06.2026).
101. Docker Documentation [Электронный ресурс]. URL: <https://docs.docker.com/> (дата обращения: 09.06.2026).
102. The JavaScript Object Notation Data Interchange Format: RFC 8259 [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обращения: 09.06.2026).
103. GZIP file format specification: RFC 1952 [Электронный ресурс]. URL: <https://www.rfc-editor.org/rfc/rfc1952> (дата обращения: 09.06.2026).